

Sistemas Embebidos y Plataforma Arduino: Un Enfoque Práctico para Electromecánicos

1. Concepto de Sistemas Embebidos: ¿Qué son y por qué son importantes?

1.1. Definiendo un Sistema Embebido: El cerebro oculto de la tecnología

Se necesita entender que un **sistema embebido** (del inglés *embedded system*) es una computadora diseñada para realizar una o varias funciones dedicadas dentro de un sistema más grande. A diferencia de una computadora de uso general (como un PC o un *smartphone*), que se adapta a muchas tareas, un sistema embebido se crea para una función específica. Se podría pensar en el microondas que tienen en casa: no se utiliza para navegar por internet ni para editar videos, sino para calentar alimentos. Ese es su propósito único, y dentro de él, hay un pequeño "cerebro" electrónico que controla el tiempo, la potencia, y la rotación del plato.

- **¿Por qué es esto relevante?** Porque en el ámbito electromecánico, se encontrarán con una multitud de máquinas y dispositivos que incorporan estos sistemas. Desde el control de un motor en una línea de producción, la gestión de temperatura en un horno industrial, hasta el sistema de seguridad de un edificio, todos utilizan sistemas embebidos. Su comprensión permite diagnosticar fallas, optimizar rendimientos y, lo más importante, diseñar nuevas soluciones.

1.2. Componentes fundamentales de un Sistema Embebido: El hardware y el software trabajando juntos

Para que un sistema embebido funcione, se necesitan dos elementos principales: el **hardware** y el **software**.

- **Hardware:** Es la parte física, los circuitos electrónicos. Esto incluye un **microcontrolador** o un **microprocesador** (que es el "cerebro" principal), memoria (para guardar instrucciones y datos), y periféricos de entrada/salida (para interactuar con el mundo exterior, como leer un sensor o activar un motor).
 - **¿Cómo se relaciona esto con lo que ya saben?** Piensen en un circuito de control básico que hayan estudiado, quizás con relés o temporizadores. Un sistema embebido reemplaza gran parte de esa lógica cableada con un componente programable, lo que ofrece muchísima más flexibilidad y capacidad.
- **Software:** Son las instrucciones programadas que el hardware ejecuta. Este software se carga en la memoria del microcontrolador y le dice exactamente qué hacer: cómo leer un sensor, cuándo encender un actuador, cómo comunicarse con otros dispositivos.
 - **¿Por qué es crucial el software?** Porque sin él, el hardware es solo un montón de componentes electrónicos. El software le da "vida" y propósito. La habilidad de programar estos sistemas es lo que les permitirá diseñar soluciones innovadoras y personalizadas.

2. Plataforma Arduino: La puerta de entrada al mundo embebido

2.1. Arduino: Un entorno de prototipado al alcance

Se necesita visualizar a **Arduino** como una plataforma de desarrollo de hardware y software de código abierto, diseñada para facilitar la creación de proyectos electrónicos interactivos. Su principal ventaja es su facilidad de uso, lo que la convierte en una herramienta ideal para estudiantes e ingenieros que se inician en el mundo de los sistemas embebidos. No se necesita ser un experto en electrónica o programación para empezar a ver resultados.

- **¿Por qué Arduino y no otro microcontrolador?** Hay muchos otros microcontroladores potentes y complejos (PICs, ARM, ESP32, etc.). Sin embargo, Arduino ofrece una curva de aprendizaje suave, una enorme comunidad de soporte, y una gran cantidad de ejemplos y librerías que simplifican tareas complejas. Es como aprender a conducir un coche con caja automática antes de pasar a uno con caja manual y más complejo.

2.2. Modelos comunes de placas Arduino: UNO y Nano

Si bien existen muchos modelos de placas Arduino, se recomienda familiarizarse con los más comunes: el **Arduino UNO** y el **Arduino Nano**.

2.2.1. Arduino UNO: El estándar de facto

El **Arduino UNO** es la placa más popular y un excelente punto de partida. Se identifica por su microcontrolador ATmega328P.

- **¿Cómo se ve?** Es una placa relativamente grande, con un conector USB tipo B (el que se usa en impresoras antiguas), pines de conexión claramente etiquetados, y un conector de alimentación.
- **¿Para qué se usa?** Es ideal para prototipado en mesas de trabajo, proyectos donde el tamaño no es una limitación, y para aprender los fundamentos. Su tamaño y robustez la hacen resistente para el uso educativo.

2.2.2. Arduino Nano: Compacto y versátil

El **Arduino Nano** es una versión más pequeña del UNO, utilizando el mismo microcontrolador ATmega328P.

- **¿Cómo se ve?** Es significativamente más compacto que el UNO, con un conector mini-USB o micro-USB. Los pines están dispuestos para ser insertados en una protoboard fácilmente.
- **¿Para qué se usa?** Su tamaño lo hace ideal para proyectos donde el espacio es limitado, como en pequeños robots, dispositivos portátiles o embebidos en espacios reducidos. Si se necesita integrar la electrónica de forma más permanente en un producto final, el Nano es una excelente opción.
 - **¿Por qué es importante tener ambos en cuenta?** Porque la elección de la placa dependerá de las necesidades específicas del proyecto. Si se está aprendiendo, el UNO es más cómodo. Si se necesita integrar en un producto, el Nano es más adecuado. Ambos utilizan la misma lógica de programación, por lo que lo que se aprenda en uno, es aplicable al otro.

3. Hardware de Arduino: Entendiendo los pines y las interfaces

3.1. Pines digitales: Encendido y apagado

Los **pines digitales** de Arduino se utilizan para leer o enviar señales digitales, es decir, solo tienen dos estados posibles: ALTO (High, 5V o 3.3V, "encendido") o BAJO (Low, 0V, "apagado").

- **¿Cómo se usan?**
 - **Salida digital (OUTPUT):** Se configuran para enviar una señal. Por ejemplo, para encender o apagar un LED, activar un relé o controlar un motor a través de un controlador.
 - **¿Por qué se configuran como salida?** Porque Arduino está "sacando" una señal. Se necesita indicarle que ese pin va a ser el origen de la señal.
 - **Entrada digital (INPUT):** Se configuran para leer una señal. Por ejemplo, para detectar si un botón está presionado o si un sensor de límite se ha activado.
 - **¿Cómo funciona la lectura?** Arduino "escucha" el voltaje en ese pin. Si detecta un voltaje alto (cercano a 5V), lo interpreta como ALTO. Si detecta un voltaje bajo (cercano a 0V), lo interpreta como BAJO.
 - **¿Por qué es importante la resistencia *pull-up*/ *pull-down* ?** Cuando un pin digital se configura como entrada, si no hay nada conectado o la conexión es "flotante", el pin puede captar ruido eléctrico y dar lecturas erróneas. Una resistencia *pull-up* conecta el pin a la alimentación (5V) internamente o externamente, asegurando que el estado sea ALTO por defecto. Al presionar un botón, se conecta el pin a masa, llevando la señal a BAJO. Una *pull-down* hace lo contrario. Se usan para asegurar un estado definido cuando no hay una señal activa.

3.2. Pines analógicos: Más que solo encendido/apagado

Los **pines analógicos** de Arduino son capaces de leer un rango continuo de valores de voltaje, no solo dos estados. En el Arduino UNO/Nano, estos pines pueden convertir un voltaje entre 0V y 5V en un valor digital entre 0 y 1023. Esto se logra mediante un **Conversor Analógico/Digital (ADC)**.

- **¿Cómo se usan?**
 - **Entrada analógica (INPUT):** Se utilizan para leer señales de sensores que no son solo "sí" o "no", sino que varían gradualmente, como un sensor de temperatura, un potenciómetro (para ajustar un valor), o un sensor de luz.
 - **¿Por qué de 0 a 1023?** Porque el ADC del microcontrolador ATmega328P tiene una resolución de 10 bits ($2^{10} = 1024$). Esto significa que puede distinguir 1024 niveles diferentes de voltaje entre 0V y 5V. Cada "paso" representa aproximadamente $5V / 1024 \approx 0.00488V$. Cuanto mayor sea el valor leído, mayor será el voltaje que se está aplicando al pin.
 - **Salida analógica (PWM):** Aunque Arduino no tiene un verdadero conversor digital/analógico (DAC) en todos sus pines, algunos pines digitales pueden simular una **salida analógica** usando **PWM (Pulse Width Modulation)**, o **Modulación por Ancho de Pulso**. Esto no es un voltaje analógico real, sino una serie de pulsos cuadrados que se activan y desactivan rápidamente. Variando el "ancho" de estos pulsos (el tiempo que están en ALTO frente al tiempo total), se puede simular un voltaje promedio.
 - **¿Cómo se entiende la simulación?** Imaginen que se quiere encender un LED a la mitad de su brillo. Con PWM, el LED no recibe 2.5V constantes. Recibe 5V por la mitad del

tiempo y 0V por la otra mitad, pero esto ocurre tan rápido que el ojo humano (o el motor, etc.) percibe un brillo (o velocidad) promedio. Esto es fundamental para controlar la velocidad de motores de corriente continua o la intensidad de luces LED.

3.3. Interfaces de comunicación: Hablando con otros dispositivos

Arduino no solo interactúa con sensores y actuadores, sino que también puede comunicarse con otros dispositivos, como computadoras, otros microcontroladores o módulos especializados.

3.3.1. Comunicación Serial (UART): El "puerto serie"

La **comunicación serial** es una de las formas más básicas y comunes de comunicación entre dispositivos. Utiliza los pines RX (Recepción) y TX (Transmisión).

- **¿Cómo funciona?** Los datos se envían bit a bit a través de un solo cable. Se necesita que ambos dispositivos estén configurados a la misma **velocidad de transmisión** (baudios) para entenderse. Es como hablar por teléfono: si uno habla demasiado rápido o demasiado lento, la otra persona no entenderá.
- **¿Para qué se usa?** Es muy útil para depurar el código (enviar mensajes a la computadora para ver qué está haciendo el programa), para interactuar con la computadora a través del Monitor Serial del IDE de Arduino, o para comunicarse con otros módulos que usan este protocolo (como módulos Bluetooth o GPS).

3.3.2. I2C (Inter-Integrated Circuit): El "bus de dos cables"

El protocolo **I2C** es un bus de comunicación que permite que múltiples dispositivos se conecten a través de solo dos cables: SDA (línea de datos) y SCL (línea de reloj).

- **¿Por qué es eficiente?** Permite conectar muchos sensores o módulos utilizando solo dos pines en el Arduino, lo que ahorra pines y simplifica el cableado. Cada dispositivo en el bus tiene una **dirección única**, lo que permite a Arduino "hablar" específicamente con uno de ellos. Es como tener una línea telefónica donde se puede llamar a diferentes personas marcando su número de extensión.
- **¿Para qué se usa?** Se usa comúnmente con sensores (temperatura, acelerómetros, giroscopios), pantallas LCD y otros módulos.

3.3.3. SPI (Serial Peripheral Interface): El "bus de alta velocidad"

SPI es otro protocolo de comunicación serial, pero generalmente más rápido que I2C. Utiliza cuatro cables principales: MOSI (Master Out, Slave In), MISO (Master In, Slave Out), SCK (Serial Clock) y SS (Slave Select).

- **¿Cuándo se prefiere SPI?** Cuando se necesita una comunicación más rápida, como con tarjetas SD, pantallas OLED o algunos tipos de sensores de alta velocidad. Se diferencia de I2C en que utiliza líneas separadas para enviar y recibir datos simultáneamente, y se necesita una línea "Slave Select" para cada dispositivo esclavo, lo que lo hace menos eficiente en pines para muchos dispositivos, pero más rápido para pocos.

4. Software de Arduino: El IDE y el lenguaje de programación

4.1. El IDE de Arduino: Tu taller de programación

El **IDE (Entorno de Desarrollo Integrado) de Arduino** es el software que se utiliza para escribir, compilar y cargar los programas (llamados "sketches") a la placa Arduino.

- **¿Cómo se usa?** Se descarga e instala en la computadora. Es sencillo, con una interfaz clara que incluye un editor de texto para el código, botones para verificar (compilar) y subir el sketch, y un **Monitor Serial** para la comunicación bidireccional con la placa.
- **¿Por qué es un IDE?** Porque integra todas las herramientas necesarias en un solo lugar, simplificando el proceso de desarrollo. No se necesita instalar un compilador por separado, ni un software para cargar el código.

4.2. Lenguaje de programación C/C++ básico aplicado a Arduino: Las bases

Los programas de Arduino se escriben en un lenguaje basado en **C/C++**. Si bien parece complejo al principio, se simplifica con la estructura y funciones propias de Arduino.

4.2.1. Estructura básica de un sketch: setup() y loop()

Todo sketch de Arduino tiene al menos dos funciones principales:

- **`**void setup() { ... }**`** : Esta función se ejecuta **una sola vez** al inicio del programa, cuando la placa se enciende o se reinicia.
 - **¿Para qué se usa?** Se utiliza para inicializar la configuración:
 - Configurar los pines como entradas o salidas (`pinMode()`).
 - Iniciar la comunicación serial (`Serial.begin()`).
 - Inicializar librerías o módulos.
 - **¿Por qué solo una vez?** Porque estas son configuraciones iniciales que no necesitan repetirse. Imaginen que al encender la luz, no se necesita reconfigurar el interruptor cada vez.
- **`**void loop() { ... }**`** : Esta función se ejecuta **repetidamente** en un ciclo continuo, después de que `setup()` ha terminado.
 - **¿Para qué se usa?** Contiene la lógica principal del programa:
 - Leer sensores.
 - Controlar actuadores.
 - Realizar cálculos.
 - Gestionar la comunicación.
 - **¿Por qué se repite indefinidamente?** Porque la mayoría de los sistemas embebidos están diseñados para operar de forma continua, respondiendo constantemente a su entorno. Un termostato no mide la temperatura una sola vez; la mide continuamente para mantenerla en el valor deseado.

4.2.2. Entrada/Salida Digital (E/S Digital): pinMode(), digitalWrite(), digitalRead()

Estas funciones son esenciales para interactuar con los pines digitales:

- **`**pinMode(pin, modo)**`** : Se utiliza en `setup()` para configurar un pin como `INPUT` (entrada), `OUTPUT` (salida) o `INPUT_PULLUP` (entrada con resistencia *pull-up* interna activada).

- **Ejemplo:** `pinMode(13, OUTPUT);` configura el pin digital 13 como salida.
- **`**digitalWrite(pin, valor)**`** : Se utiliza para escribir un valor digital (ALTO o BAJO) en un pin configurado como `OUTPUT` .
 - **Ejemplo:** `digitalWrite(13, HIGH);` enciende un LED conectado al pin 13.
`digitalWrite(13, LOW);` lo apaga.
- **`**digitalRead(pin)**`** : Se utiliza para leer el valor digital (ALTO o BAJO) de un pin configurado como `INPUT` o `INPUT_PULLUP` .
 - **Ejemplo:** `int estadoBoton = digitalRead(2);` lee el estado de un botón conectado al pin 2.

4.2.3. Entrada/Salida Analógica (E/S Analógica): `analogRead()`, `analogWrite()`

Para los pines analógicos, se utilizan:

- **`**analogRead(pin)**`** : Se utiliza para leer el valor analógico (0-1023) de un pin analógico.
 - **Ejemplo:** `int valorSensor = analogRead(A0);` lee el valor de un sensor conectado al pin analógico A0.
- **`**analogWrite(pin, valor)**`** : Se utiliza para escribir un valor PWM (0-255) en un pin que soporta PWM (marcados con una ~ en la placa).
 - **¿Por qué de 0 a 255?** Porque la resolución de PWM en Arduino es de 8 bits ($2^8 = 256$ valores, de 0 a 255).
 - **Ejemplo:** `analogWrite(9, 127);` pondría el pin 9 a un ciclo de trabajo del 50% (media intensidad).

4.2.4. Comunicación Serial: `Serial.begin()`, `Serial.print()`, `Serial.println()`

Funciones para la comunicación serial a través del puerto USB con la computadora:

- **`**Serial.begin(velocidad)**`** : Se utiliza en `setup()` para iniciar la comunicación serial a una velocidad específica (ej. 9600, 115200 baudios).
 - **Ejemplo:** `Serial.begin(9600);` inicia la comunicación a 9600 baudios.
- **`**Serial.print(dato)**`** : Envía un dato al Monitor Serial sin un salto de línea.
- **`**Serial.println(dato)**`** : Envía un dato al Monitor Serial con un salto de línea al final.
 - **¿Por qué son útiles?** Son esenciales para la depuración del código. Permiten ver los valores de las variables, los estados de los pines o mensajes personalizados para entender qué está haciendo el programa en tiempo real.

4.3. Uso de Librerías: Expansión de capacidades

Las **librerías** en Arduino son colecciones de código pre-escrito que simplifican tareas complejas o permiten interactuar con hardware específico (sensores, pantallas, etc.) sin necesidad de escribir todo el código desde cero.

- **¿Por qué usar librerías?** Ahorran tiempo, reducen errores y permiten utilizar funcionalidades avanzadas con solo unas pocas líneas de código. Es como tener un conjunto de herramientas especializadas listas para usar, en lugar de fabricar cada herramienta desde cero.
 - **Ejemplo:** Para usar una pantalla LCD, en lugar de controlar cada pin individualmente para mostrar un carácter, se incluye la librería `LiquidCrystal` y se usa una función como `lcd.print("Hola")` .

- **¿Cómo se instalan?** Se pueden instalar desde el Gestor de Librerías en el IDE de Arduino (Herramientas > Gestionar Librerías...) o descargando archivos ZIP y añadiéndolos manualmente.
- **¿Por qué es importante saber que existen?** Porque antes de intentar escribir código para un nuevo sensor o módulo, se debe buscar si ya existe una librería. Lo más probable es que sí, y eso simplificará enormemente el proyecto.

5. Desarrollo de Aplicaciones Simples con Sensores y Actuadores

Se necesita consolidar todos los conocimientos adquiridos en la práctica. A continuación, se presentan ejemplos prácticos que integran sensores y actuadores.

5.1. Ejemplo 1: Encender y apagar un LED con un botón (E/S Digital)

Este es el clásico "¡Hola Mundo!" de Arduino.

- **Componentes:**
 - Arduino UNO/Nano
 - 1 LED
 - 1 resistencia de \$220 \Omega\$ (para el LED)
 - 1 pulsador
 - 1 resistencia de \$10K \Omega\$ (para el pulsador, como *pull-down* si no se usa `INPUT_PULLUP`)
 - Cables jumper
 - Protoboard
- **Esquema de conexión:**
 - LED: Ánodo (pata larga) a pin digital 13 (a través de resistencia de \$220 \Omega\$). Cátodo (pata corta) a GND.
 - Pulsador: Un terminal a pin digital 2. El otro terminal a GND. Una resistencia de \$10K \Omega\$ entre el pin 2 y 5V (configurando `pinMode` como `INPUT` , o se podría usar `INPUT_PULLUP` y conectar el otro terminal del pulsador a GND).
- **Código (usando `INPUT_PULLUP`):**

```
const int LED_PIN = 13;    // Definimos el pin del LED
const int BOTON_PIN = 2;   // Definimos el pin del botón

void setup() {
    pinMode(LED_PIN, OUTPUT);           // Configura el pin del LED como salida
    pinMode(BOTON_PIN, INPUT_PULLUP);   // Configura el pin del botón como
    entrada con pull-up interno
    Serial.begin(9600); // Inicia la comunicación serial para depuración
}

void loop() {
    int estadoBoton = digitalRead(BOTON_PIN); // Lee el estado del botón

    if (estadoBoton == LOW) { // Si el botón está presionado (LOW porque se
    usa pull-up)
```



```

    digitalWrite(LED_PIN, HIGH); // Enciende el LED
    Serial.println("Boton presionado: LED ENCENDIDO");
} else { // Si el botón no está presionado (HIGH)
    digitalWrite(LED_PIN, LOW); // Apaga el LED
    Serial.println("Boton suelto: LED APAGADO");
}
delay(100); // Pequeña espera para evitar lecturas erráticas y para ver
los mensajes
}

```

- **¿Por qué funciona así?** Se le indica a Arduino que monitoree el pin del botón. Cuando el botón se presiona, el pin se conecta a GND (masa), y como se configuró con `INPUT_PULLUP`, su estado pasa de HIGH a LOW. El `if` detecta este cambio y activa el LED, y cuando se suelta, el estado vuelve a HIGH y el LED se apaga. Se usa `Serial.println` para ver el estado en el Monitor Serial y verificar el funcionamiento.

5.2. Ejemplo 2: Controlar el brillo de un LED con un potenciómetro (E/S Analógica y PWM)

Este ejemplo muestra cómo leer un valor analógico y usarlo para controlar una salida PWM.

- **Componentes:**
 - Arduino UNO/Nano
 - 1 LED
 - 1 resistencia de \$220 \Omega\$ (para el LED)
 - 1 potenciómetro (ej. \$10K \Omega\$)
 - Cables jumper
 - Protoboard
- **Esquema de conexión:**
 - LED: Ánodo (pata larga) a pin digital 9 (que soporta PWM) (a través de resistencia de \$220 \Omega\$). Cátodo a GND.
 - Potenciómetro: Un terminal a 5V. Otro terminal a GND. El terminal central (wiper) a pin analógico A0.
- **Código:**

```

const int POT_PIN = A0; // Pin analógico para el potenciómetro
const int LED_PIN_PWM = 9; // Pin digital con capacidad PWM para el LED

void setup() {
    pinMode(LED_PIN_PWM, OUTPUT); // Se configura el pin del LED como salida
    Serial.begin(9600); // Inicia la comunicación serial
}

void loop() {
    int valorPotenciometro = analogRead(POT_PIN); // Lee el valor del
potenciómetro (0-1023)
    Serial.print("Valor Potenciometro: ");
    Serial.println(valorPotenciometro);
}

```



```

// Mapea el valor del potenciómetro (0-1023) al rango PWM (0-255)
int brilloLED = map(valorPotenciometro, 0, 1023, 0, 255);
Serial.print("Brillo LED (PWM): ");
Serial.println(brilloLED);

analogWrite(LED_PIN_PWM, brilloLED); // Aplica el valor PWM al LED
delay(10); // Pequeña espera
}

```

- ¿Por qué se usa `map()` ? Porque `analogRead()` devuelve un valor de 0 a 1023, mientras que `analogWrite()` espera un valor de 0 a 255. La función `map()` escala proporcionalmente un rango de valores a otro. Así, cuando el potenciómetro está a la mitad (aprox. 511), `map()` lo convierte a 127, que es la mitad del brillo. Esto permite una interfaz intuitiva para controlar la intensidad.

5.3. Ejemplo 3: Sensor de temperatura LM35 y visualización en Monitor Serial (Entrada Analógica y Serial)

Este ejemplo permite leer la temperatura ambiente y mostrarla en la computadora.

- **Componentes:**
 - Arduino UNO/Nano
 - 1 sensor de temperatura LM35
 - Cables jumper
 - Protoboard
- **Esquema de conexión:**
 - LM35: Pin 1 (Vcc) a 5V. Pin 2 (Salida) a pin analógico A0. Pin 3 (GND) a GND.
- **Código:**

```

const int LM35_PIN = A0; // Pin analógico para el sensor LM35

void setup() {
  Serial.begin(9600); // Inicia la comunicación serial
}

void loop() {
  int valorSensor = analogRead(LM35_PIN); // Lee el valor analógico del
  sensor (0-1023)

  // Convierte el valor analógico a voltaje (en milivoltios)
  // Arduino lee 0-1023 para 0-5V. Cada unidad es 5000mV/1024 = ~4.88mV
  float voltajeMV = valorSensor * (5000.0 / 1024.0);

  // El sensor LM35 entrega 10mV por cada grado Celsius
  float temperaturaC = voltajeMV / 10.0;

  Serial.print("Valor RAW del sensor: ");
  Serial.println(valorSensor);
  Serial.print("Voltaje (mV): ");

```

```
Serial.println(voltajeMV);
Serial.print("Temperatura (C): ");
Serial.println(temperaturaC);
Serial.println("---"); // Separador para cada lectura

delay(1000); // Espera 1 segundo antes de la próxima lectura
}
```

- **¿Cómo se calcula la temperatura?** El sensor LM35 es un sensor de temperatura lineal que proporciona un voltaje de salida directamente proporcional a la temperatura en grados Celsius, con una relación de $10\text{mV}/^{\circ}\text{C}$. Primero, se convierte el valor leído por `analogRead` (0-1023) a un voltaje real (0-5000 mV). Luego, se divide ese voltaje entre 10 para obtener la temperatura en grados Celsius. Este ejemplo ilustra la importancia de la conversión de unidades y la interpretación de los datos del sensor.

Este material ha abordado los conceptos fundamentales de los sistemas embebidos, la plataforma Arduino, su hardware, software y aplicaciones básicas con sensores y actuadores, proporcionando una base sólida para el desarrollo de proyectos electromecánicos.

Síntesis: Los sistemas embebidos son computadoras dedicadas a funciones específicas, y Arduino es una plataforma accesible para prototipar y programar estos sistemas, permitiendo controlar hardware mediante código C/C++ y expandiendo sus capacidades con librerías para interactuar con sensores y actuadores.

Síntesis muy corta de los 5 puntos más importantes:

1. Los sistemas embebidos son cerebros dedicados para tareas específicas.
2. Arduino facilita la interacción entre hardware y software.
3. Los pines permiten interactuar digital y analógicamente.
4. El IDE de Arduino y C/C++ son el entorno de programación.
5. Las librerías y ejemplos simplifican el desarrollo de proyectos.

12760

Indicaciones Generales para entregas

Se presenta a continuación una guía para la realización y entrega de la actividad, que deberá ser llevada a cabo de forma individual.

- **Modalidad y Plazo de Entrega:** La actividad debe ser realizada individualmente y entregada al docente antes de la finalización de la clase.
 - **Formato de Presentación:** Se permite la entrega de trabajos manuscritos o electrónicos.
-

Confirmación de Entrega

Para asegurar la correcta recepción de la actividad, se recomienda registrar la entrega en el cuaderno de comunicados, redactando la siguiente constancia:

"En el día de la fecha <dd/mm/aa> , entregué actividad de Unidad: <xx> , tema: <yy> ."

Posteriormente, se necesita la firma del docente para validar la recepción.

Identificación del Trabajo

Se requiere incluir los siguientes datos para una clara identificación de cada trabajo:

- Primer apellido y primer nombre del alumno.
- Curso al que pertenece el alumno.
- Unidad a la que corresponde la actividad.
- Tema abordado.

Asimismo, en cada respuesta, se debe indicar el número de pregunta y transcribir el texto de la pregunta correspondiente.

Consideraciones Adicionales

Uso de Fuentes y Citas

No es necesario citar fuentes externas, ya que el texto base para la actividad será proporcionado por el docente.

Formato de Entrega Electrónica

- **Dirección de Envío:** Los trabajos electrónicos deben enviarse a joriza2024+tec6-71@gmail.com .

- **Formato del Archivo:** La entrega debe ser en texto plano, ya sea en el cuerpo del mensaje o como un archivo adjunto en formato `.txt`.
- **Nomenclatura del Archivo:** Se necesita que el archivo adjunto se nombre de la siguiente manera: `Apellido_Nombre_Curso_ActividadXXUnidadYY.txt`.

Legibilidad en Trabajos Manuscritos

Se necesita que la letra sea clara y legible, con un tamaño adecuado para su fácil lectura. Se recomienda utilizar tinta de color oscuro (azul o negro) y evitar tachaduras excesivas para mantener la pulcritud del trabajo.

Criterios de Evaluación

La evaluación de la actividad considerará los siguientes aspectos:

- **Claridad y Precisión:** Se valorará que las respuestas sean concisas y aborden directamente lo solicitado en cada pregunta.
 - **Coherencia y Cohesión:** Se analizará la lógica interna de las respuestas y la relación entre las ideas presentadas.
 - **Fundamentación:** Se tendrá en cuenta la capacidad para justificar las respuestas utilizando la información proporcionada por el docente.
 - **Organización y Presentación:** Se considerará el cumplimiento de los requisitos de formato y la pulcritud general del trabajo.
 - **Seguimiento de Indicaciones:** La correcta aplicación de todas las indicaciones dadas forma parte de la evaluación de los objetivos de la actividad.
-

Gestión de Preguntas o Dudas

Mientras el plazo de entrega se encuentre vigente, los alumnos pueden realizar las consultas que necesiten para clarificar cualquier aspecto de la actividad. Una vez vencido el plazo, el alumno cuenta con las instancias de intensificación para abordar dudas, completar el trabajo o realizar uno nuevo con preguntas similares.

Cuestionario:

1. ¿Cuál es la naturaleza distintiva de un sistema embebido en contraste con una computadora de propósito general, y cuál es su función principal?
2. En el contexto de la arquitectura de un sistema embebido, ¿cuáles son los dos componentes fundamentales cuya interacción es indispensable para su operatividad?
3. Desde la perspectiva del hardware de un sistema embebido, ¿qué elementos constitutivos son esenciales para su funcionalidad, incluyendo su "cerebro" principal?
4. ¿Cuál es la trascendencia del software en un sistema embebido, y qué rol desempeña en la activación del hardware?

5. ¿Qué distingue a la plataforma Arduino como una herramienta idónea para la iniciación en el desarrollo de sistemas embebidos, destacando su principal atributo?
6. En la gama de modelos de placas Arduino, ¿cuáles son los dos modelos más comúnmente recomendados para familiarización, y cuál microcontrolador comparten?
7. ¿Cuál es la función primordial de los pines digitales de Arduino y cuántos estados posibles pueden representar en la transmisión o lectura de señales?
8. En el ámbito de las entradas digitales, ¿por qué es crucial la implementación de resistencias pull-up o pull-down, y qué fenómeno previenen?
9. ¿Cuál es la capacidad distintiva de los pines analógicos de Arduino en la lectura de valores de voltaje, y cuál es el rango digital resultante de esta conversión en los modelos UNO/Nano?
10. ¿Cuál es el fundamento de la función `analogWrite()` en Arduino y cómo se simula una salida analógica real en pines que no poseen un Conversor Digital/Analógico (DAC) genuino?
11. ¿Cuáles son las dos funciones esenciales que componen la estructura básica de todo "sketch" en Arduino, y cuál es la finalidad de cada una?
12. En la programación de Arduino, ¿cuál es el propósito de la función `map()` y por qué es fundamental en el control de brillo de un LED con un potenciómetro?
13. ¿Cuál es el principio de funcionamiento del sensor de temperatura LM35 y cómo se deduce la temperatura en grados Celsius a partir de su salida analógica en un sistema Arduino?
14. En el ámbito de la comunicación entre dispositivos, ¿cuál es la característica distintiva del protocolo I2C en términos de ahorro de pines y la identificación de dispositivos?
15. ¿Cuál es la principal ventaja de utilizar librerías en el entorno de programación de Arduino y cómo contribuyen a la eficiencia y la reducción de errores en el desarrollo?